Robust Empirical Time-Complexity Inference via Scalable Benchmark Construction

Your Name
Fudan University
Email: your@email.com

***Abstract*—Empirical time-complexity inference aims to approximate a program's asymptotic growth behavior by measuring runtimes under increasing input scales and fitting candidate complexity models. While attractive for automated evaluation and benchmarking, empirical inference is often unreliable in practice due to warm-up bias, runtime outliers, and measurement noise from system-level variability. These issues become particularly visible when distinguishing closely related classes such as $O(n)$ and $O(n \log n)$.**

This paper presents a system-oriented redesign of an existing empirical complexity inference pipeline. Our approach integrates (i) warm-up elimination, (ii) median-based aggregation, (iii) finite-value filtering, and (iv) refined multiplier-based input scaling, to improve fitting stability and robustness. To evaluate at realistic scale, we construct a benchmark of over 1,000 accepted Codeforces solutions with ground-truth labels derived from official analyses and manual verification. Under identical experimental settings, our enhanced pipeline demonstrates more stable regression behavior compared to the original baseline framework, enabling more reliable complexity classification in noisy environments.

***Index Terms*—empirical complexity inference, time complexity, benchmarking, robust aggregation, log-log regression, program performance modeling**

I. Introduction

A. Motivation

Time-complexity analysis is a foundational tool in algorithm design, education, and performance engineering. In its classical form, complexity analysis relies on asymptotic reasoning and formal proofs. Such proofs are precise and general, but can be time-consuming and require expertise that does not scale to large collections of programs.

In contrast, empirical time-complexity inference seeks to infer asymptotic behavior directly from execution measurements. A typical pipeline runs the program with systematically increasing input sizes, collects runtimes, and fits the observed trend against candidate complexity classes. This approach is appealing in settings where programs are numerous, heterogeneous, or generated automatically, such as:

- automated grading systems for programming courses,
- large-scale regression testing and performance monitoring,
- benchmarking model-generated code,
- evaluation of algorithmic repositories and competitive programming code.

However, empirical inference is fragile. Real measurements are noisy, and subtle differences between complexity classes may be drowned out by runtime variability. In practical workflows, instability is not a corner case; it is the default unless the pipeline explicitly addresses measurement noise.

B. Why Empirical Inference Fails in Practice

Several sources of instability commonly arise:

- **Warm-up effects:** interpreter initialization, JIT effects (if present), module imports, and first-run cache misses can inflate early measurements.
- **Outliers:** transient scheduling events or background daemons may introduce sporadic long runs. Mean-based aggregation can be disproportionately influenced by these.
- **Non-finite or invalid samples:** timeouts, errors, or overflow conditions may produce invalid values that must be filtered.
- **Scaling sensitivity:** if input sizes grow too slowly, n vs. $n \log n$ can look similar across the measured range; if too aggressively, timeouts become frequent, reducing usable data.

These issues interact. For example, aggressive scaling increases timeout risk, producing missing values, which then disrupt fitting. Conversely, conservative scaling reduces separation between candidate growth curves, amplifying the impact of noise.

C. Key Idea

The central idea of this work is to treat empirical complexity inference as a *measurement system* rather than solely a *curve fitting problem*. This implies engineering choices that increase statistical robustness without exploding runtime cost. Specifically, we focus on:

- robust aggregation (median over repeated runs),
- systematic warm-up elimination,
- explicit filtering for non-finite measurements,
- refined input scaling to stabilize regression and model selection.

D. Benchmark at Realistic Scale

Methodological improvements must be validated on realistic workloads. Synthetic programs or small curated suites often underrepresent variability in: implementation style, input preprocessing, constant factors, and interpreter/runtime overhead. To address this, we build a benchmark from real accepted solutions in Codeforces. Ground-truth labels are derived from official editorial complexity claims and verified through manual inspection.

E. Contributions

This paper makes the following contributions:

- **Robust aggregation pipeline:** We introduce warm-up elimination, median-based consolidation, and finite-value filtering to mitigate noise and outliers in runtime measurements.
- **Scaling and selection refinements:** We refine multiplier-based scaling and stabilize regression-based model selection across candidate complexity classes, with focus on borderline cases.
- **Large-scale benchmark:** We construct a benchmark of 1,000+ real-world algorithm implementations with verified ground-truth complexity labels.
- **Reproducible evaluation protocol:** We describe a controlled evaluation setup, metrics, and ablation design for comparing the baseline pipeline against our enhanced system.

F. Paper Organization

Section **??** reviews background and problem setting. Section **??** presents the enhanced inference pipeline. Section **??** describes benchmark construction and labeling. Section **??** details experimental setup and metrics. Section **??** reports results (with placeholders where pending). Section **??** discusses threats to validity. Section **??** concludes and outlines future work.

II. Background and Problem Setting

A. Empirical Complexity Inference as a System

Empirical inference typically proceeds in three steps:

1) **Input scaling:** generate a sequence of input sizes $n_1 < n_2 < \cdots < n_k$.
2) **Measurement:** execute program P under each size n_i and record runtimes $T_j(n_i)$ for repeated runs $j = 1 \ldots m$.
3) **Model fitting:** aggregate measurements to $\hat{T}(n_i)$ and fit candidate models, selecting the best class according to a score.

While model fitting is often emphasized, the measurement and aggregation steps frequently dominate reliability. A more robust fitting stage cannot compensate for systematically biased or high-variance measurements.

B. Candidate Complexity Classes

We consider a discrete set of candidate classes:

$$\mathcal{C} = \{O(1), O(\log n), O(n), O(n \log n), O(n^2), O(n^3)\}$$

This set is configurable, and in practice it can be extended to include:

$$O(\sqrt{n}), O(n^{1.5}), O(2^n)$$

However, including very large classes can increase ambiguity when data is limited or when timeouts occur early. We focus on polynomial and near-polynomial classes, which are common in competitive programming and algorithmic implementations.

C. Log-Log Linearization

A common technique is to map measurements into log space. Let $f_c(n)$ be a predictor transformation for class c. Define:

$$x_i = \log f_c(n_i) \quad \text{and} \quad y_i = \log \hat{T}(n_i)$$

We then fit:

$$y_i = \alpha x_i + \beta + \epsilon_i$$

The goodness of fit can be evaluated via R^2 or other metrics. This approach is widely used due to simplicity and interpretability.

D. Why $O(n)$ vs. $O(n \log n)$ is Hard

A recurring failure mode is confusion between $O(n)$ and $O(n \log n)$. Over finite ranges, $\log n$ grows slowly. If input sizes are small or scaling factor γ is low, the distinction in log space may be weak relative to noise. This motivates:

- increasing input range coverage,
- careful scaling design,
- robust aggregation to reduce variance.

III. Methodology

A. Overview

We build upon the baseline empirical inference pipeline (e.g., BigOBench) and introduce robustness improvements while preserving compatibility with the overall workflow. At a high level, our pipeline consists of:

1) **Measurement Engine:** execute programs across multiple input scales.
2) **Robust Aggregation Module:** warm-up elimination, median aggregation, finite filtering.
3) **Inference Module:** candidate transformation, log-log regression, selection based on regression quality.

B. System Architecture Diagram

Figure **??** illustrates the system architecture.

C. Measurement Model and Noise

We model the j-th runtime measurement at input size n as:

$$T_j(n) = T^*(n) + \epsilon_j(n)$$

where $T^*(n)$ is an idealized "true" runtime under stable conditions, and $\epsilon_j(n)$ captures noise from system interference and runtime variance.

In practice, $\epsilon_j(n)$ is not well-modeled as Gaussian noise. We observed heavy-tailed behavior where occasional large outliers occur. In such cases, mean aggregation can be unstable.

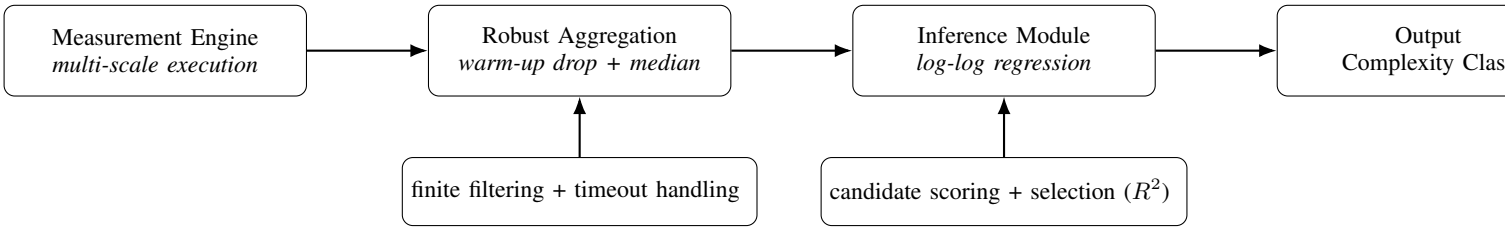

Fig. 1. Overall architecture of the enhanced empirical complexity inference framework.

D. Warm-up Elimination

Warm-up effects are common in Python due to:

- module import overhead,
- first-time bytecode paths,
- caching and allocator initialization.

We therefore discard a fixed number of initial runs at each input size:

$$\{T_j(n)\}_{j=1}^{m} \to \{T_j(n)\}_{j=w+1}^{m}$$

where w is the warm-up count. In our implementation, w is configurable.

E. Median-Based Aggregation

Given remaining measurements after warm-up elimination, we compute:

$$\hat{T}(n) = \text{median}(T_{w+1}(n), \ldots, T_m(n))$$

Median aggregation is robust to outliers. Intuitively, it selects the "typical" runtime rather than being skewed by rare slow runs.

F. Finite-Value Filtering

Empirical measurements may contain:

- non-finite values due to overflow,
- placeholders for failed runs,
- missing values due to timeouts.

We filter out invalid samples before aggregation and fitting. If insufficient valid samples remain for a scale, that scale may be excluded from regression, or the program may be marked as timeout / invalid depending on policy.

G. Input Scaling Design

We generate input sizes via geometric progression:

$$n_{i+1} = \gamma \cdot n_i$$

where $\gamma > 1$ is a scaling multiplier. A larger γ increases separation between complexity classes but increases timeout risk. A smaller γ reduces timeout risk but makes it harder to distinguish $O(n)$ from $O(n \log n)$ over finite ranges. We treat γ as a tunable parameter and report it in experimental setup.

H. Candidate Transformations

For each candidate class c, define transformation $f_c(n)$, e.g.:

$$f_{O(1)}(n) = 1 \quad f_{O(\log n)}(n) = \log n \quad f_{O(n)}(n) = n$$
$$f_{O(n \log n)}(n) = n \log n \quad f_{O(n^2)}(n) = n^2 \quad f_{O(n^3)}(n) = n^3$$

Implementation detail: we use natural log, and ensure $\log n$ is well-defined (skip $n \leq 1$ scales or shift).

I. Log-Log Regression Formulation

Let:

$$x_i = \log f_c(n_i) \quad y_i = \log \hat{T}(n_i)$$

We fit:

$$y_i = \alpha x_i + \beta + \epsilon_i$$

where α is an estimated slope and β is an intercept. We compute goodness of fit via:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

We then select:

$$c^* = \arg\max_{c \in \mathcal{C}} R_c^2$$

J. Selection Heuristics and Stability Checks

In addition to pure R^2 ranking, we incorporate stability checks:

- minimum required number of valid scales ($k_{\min}$),
- rejection of regressions with degenerate variance,
- optional tie-breaking for near-equal R^2 values.

These checks reduce pathological selections under sparse data or noisy regimes.

K. End-to-End Procedure

Algorithm **??** summarizes the full pipeline.

L. Implementation Notes

Our implementation is designed for compatibility with the baseline framework. Key engineering choices:

- perform aggregation at worker-side to minimize downstream changes,
- store both raw measurements and aggregated summaries when possible,
- ensure backward compatibility by falling back to raw value lists if aggregate fields are missing,
- isolate policy choices (timeouts, warm-up count, scaling multiplier) into configuration.

Algorithm 1 Robust Empirical Complexity Inference

Require: Program P, input sizes $\{n_1, \ldots, n_k\}$, repeats m, warm-up w
Ensure: Inferred class c^* or TIMEOUT/INVALID

```
for i = 1 to k do
    Run P on size n_i for m repeats, collect {T_j(n_i)}_{j=1}^m
    Discard warm-up runs: keep {T_j(n_i)}_{j=w+1}^m
    Filter invalid values (non-finite, error markers)
    if valid samples below threshold then
        mark scale as unusable
    else
        T̂(n_i) ← median(valid samples)
    end if
end for
if insufficient usable scales then
    return INVALID
end if
for each candidate class c ∈ C do
    compute x_i ← log f_c(n_i) for usable scales
    compute y_i ← log T̂(n_i) for usable scales
    fit y = αx + β by linear regression
    compute R_c^2
end for
c* ← arg max_c R_c^2
return c*
```

IV. Benchmark Construction

A. Dataset Goals

The benchmark is designed to stress empirical inference under realistic diversity:

- heterogeneous implementation styles,
- varying constant factors and I/O preprocessing,
- diverse algorithmic paradigms,
- real constraints that induce different runtime regimes.

B. Source and Collection

We collect accepted solutions from Codeforces. For each selected problem, we retrieve:

- problem identifier,
- solution code (Python subset in current draft),
- metadata (difficulty, tags if available),
- official/editorial complexity claims when accessible.

C. Inclusion Criteria

We include solutions that satisfy:

- accepted verdict,
- deterministic behavior on generated inputs,
- runnable under the evaluation harness,
- amenable to input scaling (we can generate larger inputs consistent with the problem structure).

D. Complexity Labeling

Ground-truth complexity labels are assigned using a hybrid procedure:

TABLE I
BENCHMARK COMPLEXITY DISTRIBUTION (PLACEHOLDER)

Class	Count	Fraction
$O(1)$	XX	XX
$O(\log n)$	XX	XX
$O(n)$	XX	XX
$O(n \log n)$	XX	XX
$O(n^2)$	XX	XX
$O(n^3)$	XX	XX

1) **Official label:** editorial or official analysis provides intended complexity class.
2) **Manual verification:** inspect representative implementations and confirm the main algorithmic structure matches the claimed class.
3) **Ambiguity handling:** when multiple strategies exist, prefer the actual implementation's structure rather than the problem's theoretical best.

This procedure reduces label noise, especially for problems where multiple accepted solutions differ in complexity.

E. JSONL Format

Each instance is stored as one JSON line with fields:

- `problem_id`: unique ID,
- `solution_id`: unique ID,
- `solution_code`: source code,
- `time_complexity`: ground-truth label,
- optional metadata fields.

Example (schematic):

```
{"problem_id": "...", "solution_id": "...",
 "solution_code": "...",
 "time_complexity": "O(n log n)"}
```

F. Dataset Statistics (Placeholder)

We report dataset summary statistics:

- number of programs: $N = XX$ (expected > 1000),
- distribution over complexity classes: Table **??**,
- distribution over tags/difficulty: [TODO: optional].

V. Experimental Setup

A. Environment

All experiments are conducted in a controlled Linux environment. We fix:

- Python version: XX,
- OS distribution: XX,
- CPU model: XX,
- RAM: XX,
- CPU frequency scaling policy: [TODO: optional],
- process isolation policy: [TODO: optional].

We emphasize that absolute runtimes are not the target; relative growth trends are. Nevertheless, stability benefits from controlling background load.

TABLE II
OVERALL PERFORMANCE COMPARISON (PLACEHOLDER)

Method	Accuracy (%)	Mean R^2	Non-timeout
Baseline	XX	XX	XX
Ours	XX	XX	XX

B. Execution Policy

For each program:

- number of scales: $k = XX$,
- scaling multiplier: $\gamma = XX$,
- repeats per scale: $m = XX$,
- warm-up runs discarded: $w = XX$,
- per-run timeout: XXseconds,
- global per-program timeout: XXseconds.

We use identical policies for Baselineand Oursto ensure fair comparison.

C. Metrics

We evaluate:

- **Accuracy:** fraction of programs where predicted class matches ground truth.
- **Mean R^2:** average regression goodness-of-fit score across programs (computed for selected class).
- **Non-timeout ratio:** fraction of programs that complete all scales without timeout.

We also define optional diagnostics:

- confusion matrix over classes,
- per-class precision/recall,
- stability variance across repeated benchmark runs.

D. Baselines

Baselineis the original BigOBenchpipeline without our robustness changes. Oursincludes:

- warm-up elimination,
- median aggregation,
- finite filtering,
- scaling refinement / selection stability checks.

E. Ablation Design

We conduct an ablation study to isolate each component:

1) Baseline
2) Baseline+ warm-up elimination
3) Baseline+ warm-up elimination + median aggregation
4) full Ours(add finite filtering + scaling refinement)

This design ensures improvements can be attributed rather than aggregated into a single opaque change.

VI. RESULTS

A. Overall Comparison

Table **??** summarizes overall performance.

We expect Oursto improve stability and accuracy primarily by reducing aggregation noise and regression sensitivity. We avoid asserting numerical gains until experiments complete.

TABLE III
PER-CLASS ACCURACY (PLACEHOLDER)

Class	Baseline	Ours
$O(1)$	XX	XX
$O(\log n)$	XX	XX
$O(n)$	XX	XX
$O(n \log n)$	XX	XX
$O(n^2)$	XX	XX
$O(n^3)$	XX	XX

TABLE IV
ABLATION STUDY (PLACEHOLDER)

Variant	Acc.	Mean R^2	Non-timeout
Baseline	XX	XX	XX
+ warm-up	XX	XX	XX
+ median	XX	XX	XX
Full (Ours)	XX	XX	XX

B. Per-Class Breakdown

We include a per-class breakdown (placeholder table).

C. Borderline Cases: $O(n)$ vs. $O(n \log n)$

A key stress test is separating $O(n)$ from $O(n \log n)$. In practice, failures occur when:

- input ranges are not large enough for $\log n$ to noticeably vary,
- noise variance is large relative to the marginal difference,
- timeouts remove higher-scale points, weakening regression.

Oursaddresses these by:

- reducing variance via median aggregation,
- improving usable points via finite filtering policies,
- stabilizing selection via additional checks.

D. Ablation Results (Structure)

We report ablation outcomes using Table **??**. Values are placeholders until runs complete.

E. Stability Diagnostics

Beyond accuracy, we examine fitting stability:

- variance of estimated slope α across repeated runs,
- sensitivity of selected class when a single scale is removed,
- distribution of R^2 for selected models.

We include these as optional analyses if time permits.

F. Error Analysis

We categorize misclassifications:

- **Class adjacency errors:** $O(n)$ vs. $O(n \log n)$, $O(n \log n)$ vs. $O(n^2)$.
- **Timeout-induced errors:** high-scale points missing leads to underestimation.
- **Label ambiguity errors:** multiple valid strategies exist but label reflects editorial not implementation.
- **Harness mismatch errors:** input generator fails to mimic intended scaling regime.

We recommend reporting top-K error exemplars with short code-level reasoning.

VII. Discussion

A. Why Robust Aggregation Matters

Empirical inference relies on finite measurements that are affected by noise. Median aggregation reduces the influence of heavy-tailed outliers. Warm-up elimination reduces systematic bias that can distort early scale points, which are disproportionately influential when k is small.

B. Scaling Trade-offs

Input scaling is a balancing act:

- too slow $\Rightarrow$ classes collapse under noise,
- too fast $\Rightarrow$ frequent timeouts reduce usable points.

A robust pipeline should tolerate moderate scaling choices by avoiding brittle dependence on any single point.

C. Dataset Labeling Challenges

Using official complexity claims improves credibility but introduces mismatch: editorials may assume optimal algorithm, while actual submitted code may have additional factors or different approach. Manual verification helps, but cannot eliminate all ambiguity. We treat labeling as a first-class engineering step, not an afterthought.

D. When Empirical Inference is Appropriate

Empirical inference cannot replace formal proofs. However, it is valuable for:

- regression detection (performance changes over time),
- educational feedback (approximate class guidance),
- benchmarking model-generated code at scale.

VIII. Threats to Validity

A. Internal Validity

Measurement noise may still affect regression outcomes. We mitigate this via repeated runs, warm-up elimination, and robust aggregation. Hardware variation across machines can cause different noise regimes; we reduce this by running in a controlled environment.

B. Construct Validity

Ground-truth labels may be incorrect or ambiguous. We reduce this risk by combining official analyses with manual verification and by explicitly tracking ambiguous cases as a separate bucket if needed.

C. External Validity

The benchmark is derived from competitive programming code. While diverse, it may not represent all production workloads, particularly systems code with complex I/O or concurrency. Future work could extend to broader repositories.

D. Conclusion Validity

If experiments are incomplete at the time of writing, reported results should be treated as preliminary. We recommend updating all placeholders before submission and archiving the final configuration.

IX. Conclusion

This paper presented a robust empirical time-complexity inference pipeline that improves measurement stability through warm-up elimination, median aggregation, finite-value filtering, and refined scaling and selection procedures. We also constructed a 1,000+ program benchmark from Codeforces with verified ground-truth labels, enabling large-scale evaluation under realistic program diversity.

Future work includes:

- adaptive scaling that chooses γ based on early measurements,
- confidence intervals or uncertainty estimates for predicted class,
- hybrid inference combining static analysis signals with empirical fitting,
- expansion of benchmarks beyond competitive programming.

Appendix A
Reproducibility Checklist

This appendix provides a practical checklist for reproducing experiments.

A. Environment

- OS: [TODO: fill]
- Python: [TODO: fill]
- CPU: [TODO: fill]
- RAM: [TODO: fill]
- Kernel: [TODO: fill]
- CPU governor: [TODO: fill]

B. Configuration

- number of scales k: [TODO: fill]
- multiplier γ: [TODO: fill]
- repeats m: [TODO: fill]
- warm-up w: [TODO: fill]
- timeout policy: [TODO: fill]

C. Dataset

- dataset name: Codeforces-Complexity-1K
- size: [TODO: fill]
- label source: official + manual verification

Appendix B
Dataset Schema

A. Fields

We recommend the following schema fields per record:

- `problem_id` (string)
- `solution_id` (string)
- `solution_code` (string)
- `time_complexity` (string)
- `language` (string)
- `tags` (list of strings) [optional]
- `difficulty` (int) [optional]
- `source_url` (string) [optional]
- `label_source` (string: editorial/manual/both) [optional]

B. Example

```
{
  "problem_id": "1717_300A",
  "solution_id": "1717_238",
  "language": "python",
  "time_complexity": "O(n log n)",
  "solution_code": "..."
}
```

Appendix C
Confusion Matrix Template

This template can be filled once results are available. Rows are ground-truth classes; columns are predicted classes.

TABLE V
CONFUSION MATRIX (PLACEHOLDER)

GT \ Pred	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(n^3)$
$O(1)$	XX	XX	XX	XX	XX	XX
$O(\log n)$	XX	XX	XX	XX	XX	XX
$O(n)$	XX	XX	XX	XX	XX	XX
$O(n \log n)$	XX	XX	XX	XX	XX	XX
$O(n^2)$	XX	XX	XX	XX	XX	XX
$O(n^3)$	XX	XX	XX	XX	XX	XX

Appendix D
Expanded Notes for Drafting

A. Pending Tasks

- Fill environment specs in Section **??**.
- Replace all XXentries with real numbers.
- Add citations for baseline framework and robust statistics.
- Decide final candidate set $\mathcal{C}$ for submission.
- Add at least one real-world case study example (optional).
- Ensure the dataset release plan is consistent with policy.
- Add repository link in camera-ready version (if allowed).
- Add training/validation split if needed for downstream tasks.

B. Writing Style Notes

Avoid over-claiming. Use concrete, testable statements. Prefer: "We observe" / "We find" / "We measure". Avoid: "This proves" unless mathematically proven. Use short sentences for clarity. Keep paragraphs focused on one point. Minimize buzzwords.

C. Implementation Notes

Aggregation: store raw samples. store aggregated medians. keep warm-up count explicit. record timeouts. record skipped scales. record final usable scales.

Scaling: document base size n_1. document multiplier γ. document max size n_k. ensure input generator is stable.

Regression: handle degenerate variance. handle small sample sizes. handle log transform invalid points. tie-break with secondary signals if needed.

D. Ablation Notes

Warm-up elimination alone: should reduce early-scale inflation. Median alone: should reduce outlier impact. Finite filtering: should reduce failure contamination. Scaling refinement: should reduce confusion among adjacent classes.

E. Result Narrative Template

Once numbers are available, use a narrative like:

- Overall accuracy improves from A to B.
- Mean R^2 increases, indicating more stable fits.
- Non-timeout ratio improves due to reduced unstable scaling / better policy.
- Biggest gain is in $O(n)$ vs. $O(n \log n)$ separation.
- Remaining errors cluster in adjacency confusions.